

Progress report: Speeding up motif search

Tarek Tohme

July 30, 2019

1 Introduction

This report summarizes attempts made to speed up subgraph isomorphism in the context of network motif search. Many networks of interest in scientific literature are sparse, or have power-law or fat-tailed degree distributions, namely protein-protein interaction networks, social networks, and the internet (add references). We take advantage of these practical assumptions to count instances of a graph H in a network G faster than would be possible otherwise. Our work improves on the Grochow-Kellis subgraph querying algorithm (add reference), and takes advantage of the onion decomposition proposed in (add reference). By onion-decomposing H and G , we impose early aborts on isomorphism testing using a vertex's coreness and onion layer. We also use fast algorithms to count instances of trees in H 's 1-shell isomorphic to subtrees in G 's 1-shell, which, under our assumptions should comprise a significant portion of G . Finally, a dynamic variation of the ISOMORPHICEXTENSIONS routine proposed in (add reference), coupled with the onion decomposition, offers a further performance improvement. We assume the reader to be familiar with the above referenced papers and the k-core decomposition terminology.

2 The onion decomposition

The onion decomposition removes all vertices of degree at most k from a graph $G = (V, E)$, one pass at a time, starting from $k = 1$. At each pass, it labels the removed vertices with the index of the pass, called the onion layer, and the coreness of the vertex, as defined by the k-core decomposition (add source).

2.1 Early abort by onion layer and coreness

If there exists a subgraph $H' = (V_{H'}, E_{H'})$ of G isomorphic to H , it is easy to see that an onion decomposition on G cannot label a vertex $v' \in V_{H'}$ with an onion layer less than that of its image $v \in V_H$. The subgraph isomorphism algorithms discussed in this report proceed by extending a partial mapping

between H and G one node at a time (see section 3); we can therefore rule out the possibility of isomorphism early on in the process if some vertex in H has a greater onion layer than the node it would map to in G .

We can also use coreness for aborting a candidate isomorphism early. First notice that a vertex in a graph's k -shell can have at most k neighbors in the $k + 1$ -core. Otherwise, it wouldn't be removed from the k -shell when the k -core decomposition (or the onion decomposition) is making passes to remove nodes of degree at most k , and hence would not be in the k -shell. By the same reasoning, a vertex in the $k + 1$ -core must have at least $k + 1$ neighbors in the $k + 1$ -core. Because of this, mapping a vertex in H 's $k + 1$ -core to a vertex in some shell of G with coreness less than k violates the conditions for induced isomorphism.

2.2 Counting subtrees in the 1-shell

The 1-shell of a graph consists of trees each having one vertex in the 2-core. Performing an onion decomposition on H and G reveals these trees, if they exist, and enables us to run them through a subtree-counting algorithm that counts instances of a query tree T in a larger tree S much faster than the counting algorithm for generic subgraph isomorphism.

2.2.1 Using matrix permanents

Here we show that $\# \text{ROOTEDSUBTREEISOMORPHISM}$, the problem of counting subtrees of a rooted unlabeled tree S that are isomorphic to a rooted unlabeled tree T is $\# \text{P}$ -complete via $\# \text{BIPARTITEMATCHINGS}$, the problem of counting matchings in a bipartite graph $B = (R, L, E)$ that map all of R to a subset of L . $\# \text{PERFECTMATCHINGS}$ is a special case of this problem, with $|R| = |L|$, and was proved $\# \text{P}$ -complete in Valiant (add source).

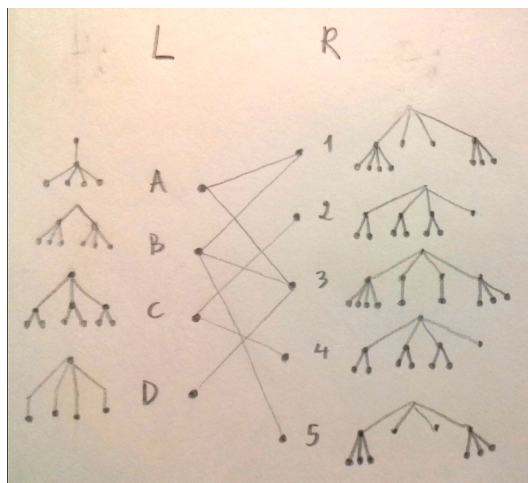


Figure 1

THEOREM 1: $\# \text{ROOTEDSUBTREEISOMORPHISM}$ is $\# \text{P}$ -complete.

PROOF: Membership in $\# \text{P}$ is easy to verify: consider a nondeterministic Turing machine that removes leaves from S nondeterministically and checks for tree isomorphism with T in polynomial time using preorder strings (add source). The number of accepting configurations is equal to the number of instances of T in S .

To prove that $\# \text{BIPARTITEMATCHINGS} \leq_T \# \text{ROOTEDSUBTREEISOMORPHISM}$, we construct two sets of trees, one for each side of a given bipartite graph $B = (R, L, E)$. Each vertex of B corresponds to a tree, as shown in Figure 1. A tree corresponding to $l \in L$ is constructed to be isomorphic to a subtree of a tree corresponding to $r \in R$ if and only if $(l, r) \in E$. Let $k = |L|$. Let L 's vertices be indexed by $i \in \{1, \dots, k\}$ and R 's vertices by $j \in \{1, \dots, |R|\}$. For each l_i , construct a perfect tree T_i of depth 2 with i children at depth 1 and $k - i + 1$ children at depth 2 for each depth-1 child. For each vertex $r_j \in R$, construct a “blank” depth-1 tree S_j with k children. Then for each vertex $l_i \in L$, if there is an edge (l_i, r_j) in E , add sets of $k - i + 1$ children at i distinct depth-1 nodes of S_j . If there are already some depth-1 nodes with more than $k - i + 1$ children in S_j , then start by counting them as sets and continue adding remaining sets elsewhere. This priority rule guarantees that there cannot be more than i nodes with at least $k - i + 1$ children in S_j .

Consider a tree T_i such that $(l_i, r_j) \in E$. By construction, S_j has exactly i sets of at least $k - i + 1$ children, so $T_i \subseteq S_j$. Now consider another tree T_m such that $(l_m, r_j) \notin E$ for some $m > i$. Since S_j has exactly i sets of at least $k - i + 1$ depth-2 children, and because the priority rule mentioned above says that larger sets of depth-2 nodes absorb smaller ones, it cannot contain an instance of T_m . If $m < i$, S_j would need m sets of $k - m + 1 > k - i + 1$ children to fit an instance of T_m . This cannot happen unless some edge (l_n, r_j) , $n < m$, violates the priority rule, and so $T_m \not\subseteq S_j$, which concludes the proof. These arguments are easy to verify visually using Figure 1.

Given rooted trees T and S of depth 2 or greater, we can recursively count the number of instances of T in S by constructing B , assigning the number of instances of T_i in S_i as the weight of edge (l_i, r_j) , and computing the permanent of its biadjacency matrix. One obstacle we still have to face is symmetry-breaking.

2.2.2 Using bipartite matching elimination

The following is an alternative to computing the permanent for counting bipartite matchings that takes advantage of some structure in the bipartite graph. The main idea is that the number of matchings is equal to the number of assignments that contain conflicting (adjacent) edges subtracted from the number of unconstrained assignments of the vertices in R to a subset of L . We can compute the number of matchings ruled out by conflicts quickly. The tricky part is correcting for assignments that are ruled out by more than one conflict.

2.3 Ear decomposition in the 2-core

The ear decomposition (add source) was suspected to enable speed improvements compared to generic subgraph isomorphism.

3 Dynamic algorithm for finding isomorphic extensions

The ISOMORPHICEXTENSIONS routine proposed in Grochow and Kellis' algorithm works by recursively extending a given partial map $f : D \mapsto R$ between H and G while preserving induced isomorphism, where $D \subset V_H$ and $R \subset V$. First, from the neighborhood of D , it selects the most constrained vertex m in V_H ; among vertices with the most already mapped neighbors, it chooses the vertices with highest degree and among those, the one with highest degree sequence. This is done to maximize the chances of ruling out non-isomorphic mappings early on in the recursion. Then, it goes through all neighboring vertices of R in V and tests for isomorphism with m on each one.

Profiling a python implementation of this function showed that it spends most of its time computing the most constrained node of H , and looping through all neighboring vertices of f 's range testing for isomorphism. Here we show how the latter task can be accomplished much faster with a dynamic algorithm, and the former becomes unnecessary with a little help from the onion decomposition.

First, note that the definition of induced isomorphism imposes that any candidate vertex n in V for an isomorphic extension of f must have exactly the same number of neighbors with R as m has with D , that number being denoted k . Therefore, we can restrict the search for potential candidates for an extension to the subset of the neighborhood of R in V where vertices have exactly k adjacent vertices in R . We call that subset of R 's neighborhood the k -club of R , denoted $C_k(R)$, where $\bigcup_{i \leq |R|} C_i(R) = N(R)$. Then, we would have to verify that these k adjacent vertices are the images of the neighbors of m in D , and that would be sufficient for induced isomorphism. We can compute $C_k(R)$ by memoization using the following equation, which is the basis for the dynamic algorithm:

$$C_{k+1}(R \cup \{v\}) = C_{k+1}(R) \cup (C_k(R) \cap N(v)) \setminus (C_{k+1}(R) \cap N(v)) \quad (1)$$

Note that if k is the maximum number of adjacent edges R has with any vertex in its neighborhood, then $C_{k+1}(R) = \emptyset$ and the above equation reduces to the middle term. (Figure 1) shows how these values would be stored in a table, just for illustration purposes. The algorithm stores these values in a tree, as will be elaborated in the following section.

3.1 Dynamic search tree

The dynamic algorithm proceeds iteratively. First, a tree data structure is initialized with a given vertex g of V as its root (for clarity, vertices refer to elements of V , and nodes refer to elements of the tree). Each node of the tree is indexed by an ordered list of the vertices of V traversed by the algorithm up to v , the most recently added node. It is therefore a search tree where nodes represent paths of successive isomorphic extensions in V . Each node also stores a row of the table shown in (Figure 1). At each iterative step, leaves of that tree represent the R 's of all valid partial mappings that have been found so far. The algorithm loops through the leaves, and for each one, computes values of $C_i(R), i \in \{1, \dots, l\}$, from the elements stored in its parent nodes, using equation (1). These values are inputted to the node's own row. By knowing k , the number of neighbors of m in D , we read $C_k(R)$ from our newly computed row elements and that represents the candidates for isomorphic extensions in V . Computing the row elements $C_i(R \cup \{v\})$ takes $O((|C_i(R)| + |N(v)|) \times l)$ using dictionary-like implementations of sets. Here l represents the maximum number of neighbors a candidate node v can have with a partial range R . As we will see in the following section, traversing the nodes of H and G a certain way gives a good upper bound on the cost of computing the k -club row elements.

3.2 Taking advantage of the onion decomposition

Since the onion decomposition works by successively removing vertices of degree at most k from a graph, traversing vertices in reverse onion-layer order guarantees that every encountered vertex has at most k neighbors with the set of vertices traversed so far. By traversing H in reverse onion layer, we impose an upper bound on l equal to the highest value of the coreness of H 's vertices, which according to the assumptions made in the introduction, won't be too intimidating. Also, the computation of most constrained vertices is now unnecessary, since we can know in advance the path in which to traverse H to extend then partial mapping's domain.

What's more, following up on our discussion in section 2.1, if we traverse the vertices of G in onion-layer order, we guarantee the most early aborts by onion layer and coreness, saving time and memory in unnecessary isomorphism tests.

4 Results

(Coming soon: speed comparison of GK, GK + #subtrees, GK-dynamic, GK-dynamic + #subtrees)

5 Remaining work

5.1 Finishing current implementations and profiling

At the time of this writing the Grochow-Kellis algorithm was implemented in Python and passes all provided test cases. However, it fails to run correctly on trees. The subtree counting algorithm was implemented and tested. It works correctly, albeit without symmetry-breaking. The dynamic Grochow-Kellis algorithm is in the debugging phase. All algorithms are expected to be ready for profiling two weeks from the date on this report.

5.2 Faster symmetry-breaking checking

The symmetry-breaking technique developed in Grochow-Kellis (add source) guarantees that instances of H in G are only found once by ruling out symmetric instances as they are encountered. However, it could still take $O(|V_H|)$ steps to recognize an instance symmetric to one we've already found. A moderate speedup could be achieved if the vertices of H were traversed in a certain order. Namely, some vertices have more symmetry-breaking conditions they are part of than others (see Grochow-Kellis). By sorting the vertices in reverse topological order, where an order relation is a symmetry-breaking condition between two vertices, we would eliminate symmetries in the fastest possible way, starting with the “most constrained” vertices.

5.3 C implementation and software documentation

The algorithms discussed in this report were all implemented in Python. In the (near) future, they will be translated to C and documented appropriately, for possible release as a software library.